DAY 5

Git & GitHub for Project Managers

Learning Objectives:

- Understand why PMs need Git and GitHub knowledge even without writing code
- Learn Git basics: repository, commit, branch, pull request, merge
- Understand how development teams use GitHub: issues, milestones, project boards
- Use GitHub to manage and track project documentation, not just code
- **YouTube Search Terms:**
  - "Git and GitHub explained for non-developers"
  - "GitHub for project managers tutorial"
  - "GitHub issues and project boards"
  - "Git branching strategy explained"
- **Recommended Channels:**
  - Programming with Mosh
  - Tech With Tim
  - Traversy Media

Tasks:

- Install Git and create a GitHub account — create a repository called pm-internship-[yourname]
- Learn the core Git commands by running them yourself: git init, git add, git commit, git push, git pull, git branch, git checkout — understand what each one does and when a PM would need to know it
- Explore GitHub features that PMs use directly — Issues for tracking bugs and tasks, Milestones for sprint or release targets, Project Boards for Kanban-style task tracking, and Pull Requests for how code changes are reviewed and approved
- Upload your Day 3 BRD and Day 4 PRD documents to your GitHub repository in a folder called /documents — write meaningful commit messages explaining what you added
- Create 5 GitHub Issues for the hypothetical app project — each issue should have a title, description, label (bug/feature/documentation), and be linked to a milestone you create called "Sprint 1"

Deliverable

- GitHub repository link shared with trainer
- Screenshot of repository with documents folder and commits
- Screenshot of 5 GitHub Issues linked to Sprint 1 milestone
- Short note: how a PM uses GitHub differently from a developer

Why PMs Need Git & GitHub

A Project Manager may not write code, but GitHub is where development work happens. PMs use it to:

- Track tasks and bugs
- Monitor development progress
- Manage releases and sprints
- Review documentation
- Coordinate between teams
- Understand what changes are being made

GitHub acts as a project tracking and collaboration platform, not just a coding tool.

Core Git Concepts

Term	Meaning
Repository (Repo)	A project folder stored on GitHub
Commit	A saved version of changes
Branch	A separate workspace for making changes
Pull Request (PR)	Request to merge changes into main project
Merge	Combining changes from one branch into another

Git Commands Learned

Command	Purpose	When a PM Might Use/Know It
`git init`	Creates a new Git repository	Understand how a project repository is created
`git add .`	Stages all changed files for commit	Before saving documentation updates
`git commit -m "message"`	Saves a snapshot of changes with a description	Track document revisions with meaningful messages

`git push`	Uploads local changes to GitHub	Publish updated documents to the repository
`git pull`	Downloads the latest changes from GitHub	Ensure local files are up to date with team changes
`git branch`	Creates or lists branches	Understand how teams work on features separately
`git checkout branch-name`	Switches between branches	View or review work being done in different branches
`git merge`	Combines changes from one branch into another	Understand how completed work is integrated
`git status`	Shows modified, staged, and untracked files	Check the current state of the repository
`git log`	Displays commit history	Review project changes and documentation updates over time
`git clone repository-url`	Copies an existing repository to a local machine	Access an existing project repository
`git remote add origin repository-url`	Connects a local repository to GitHub	Link a new project to GitHub for collaboration

GitHub Features PMs Use

1. Issues are used for:

- Features
- Bugs
- Documentation tasks
- User stories

2. Milestones help track completion of a sprint or release.

3. Project Boards

Kanban-style workflow:

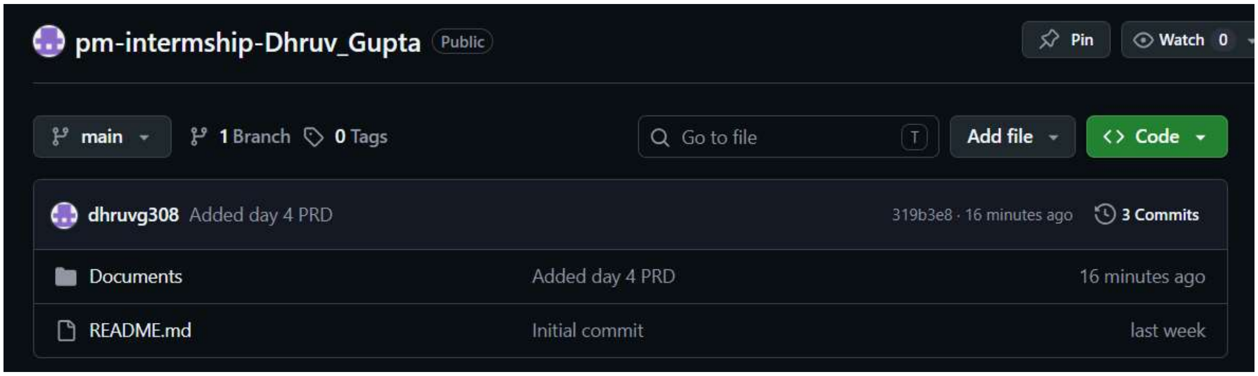
To Do
↓
In Progress
↓
Testing
↓
Done

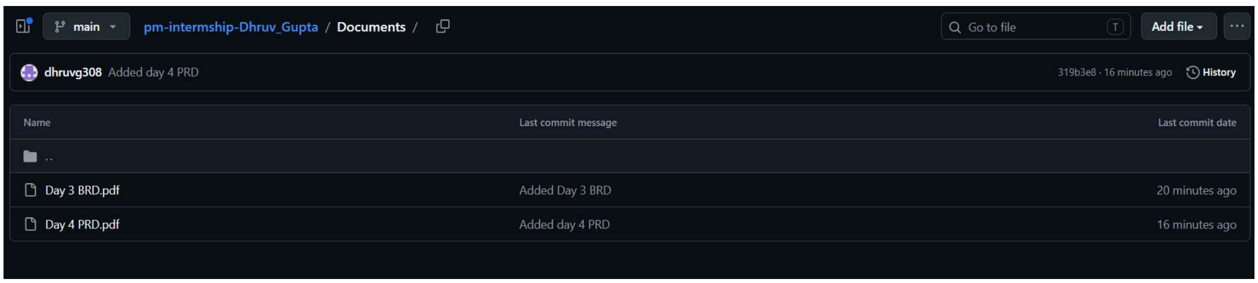
PMs use boards to track team progress visually.

4. Pull Requests is how developers request approval before code is merged.

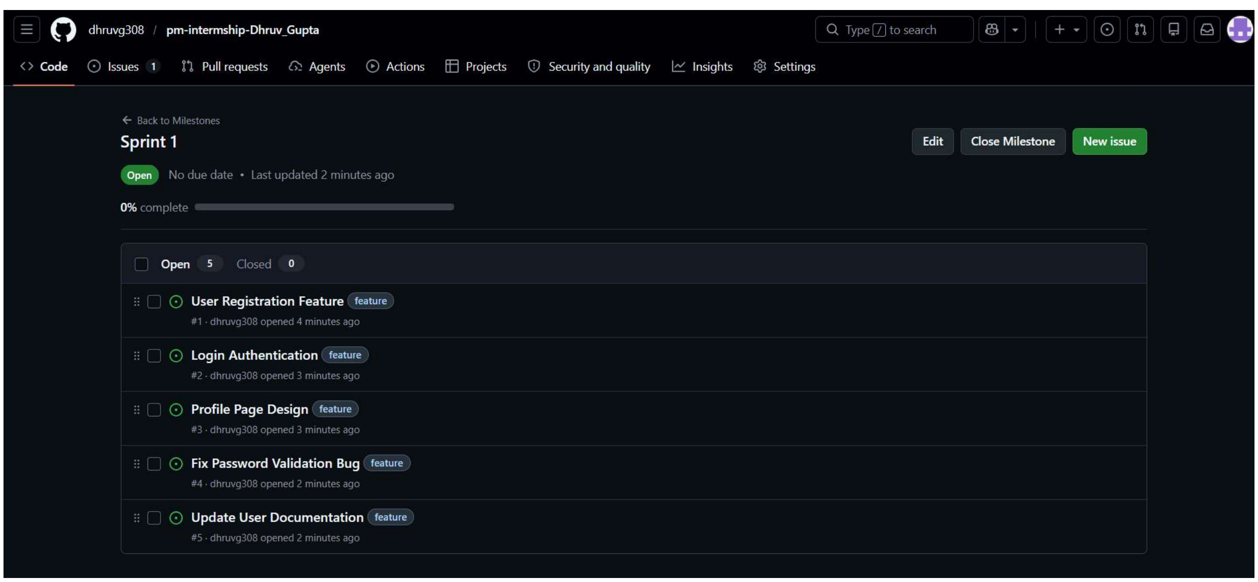
- Review PR status
- Ensure requirements are completed
- Track deployment readiness

Deliverables:





(Screenshot of repository with documents folder and commits)



(Screenshot of 5 GitHub Issues linked to Sprint 1 milestone)

Short Note: How a PM Uses GitHub Differently from a Developer

A Project Manager uses GitHub primarily for planning, tracking, collaboration, and communication rather than writing code. PMs rely on GitHub to gain visibility into project progress, coordinate work across teams, and ensure that development efforts align with business goals and product requirements. They create and manage Issues to track features, bugs, and tasks, use Milestones to organize work into sprints or releases, and maintain Project Boards to monitor the status of ongoing work. By regularly reviewing these tools, PMs can identify blockers, prioritize tasks, and keep stakeholders informed about project progress.

In addition to task management, PMs use GitHub as a central repository for important project documentation such as Business Requirement Documents (BRDs), Product Requirement Documents (PRDs), meeting notes, release plans, and user stories. This ensures that all team members have access to the latest information and can collaborate effectively. PMs may also review Pull Requests to verify that completed work matches the agreed requirements and to understand when features are ready for testing or deployment.

Developers use GitHub mainly for technical and coding-related activities. Their responsibilities include creating branches for new features or bug fixes, writing and updating code, committing changes, resolving merge conflicts, conducting code reviews, and submitting Pull Requests for approval. Developers focus on implementing functionality, maintaining code quality, and ensuring that software changes are integrated smoothly into the project.

While developers concentrate on building the product, PMs focus on managing the overall workflow, facilitating communication between stakeholders and the development team, and ensuring that deadlines and project objectives are met. PMs often use GitHub dashboards, issue reports, and milestone tracking to measure team performance and monitor project health without needing to contribute code directly.

GitHub helps both PMs and developers collaborate effectively by providing a single platform for tracking tasks, managing documentation, reviewing changes, and monitoring project progress. It creates transparency across the team, improves accountability, and enables better decision-making throughout the product development lifecycle. By understanding GitHub, a Project Manager can communicate more effectively with technical teams, track development activities with confidence, and contribute to the successful delivery of products and features.